CSE563: Unit V Study Notes

Algorithm Complexity Analysis

School of Computer Science & Engineering

Session 2025–26

Contents

1 ⏱ Unit V: Algorithm Complexity Analysis **2**
- 1.1 Measuring Running Time . . . 2
- 1.2 Asymptotic Notations . . . 2
- 1.3 Recurrence Relations . . . 3
- 1.4 The Master Theorem . . . 5
- 1.5 P, NP and NP-Completeness . . . 5
- 1.6 The Satisfiability Problem (SAT) . . . 6
- 1.7 Searching and Sorting . . . 7
  - 1.7.1 Searching . . . 7
  - 1.7.2 Sorting . . . 8
- 1.8 Unit V Quick-Revision Sheet . . . 9

1 Unit V: Algorithm Complexity Analysis

Course Outcome CO5: determine the algorithmic complexity of various mathematical problems.

Two programs that both "work" can differ by a factor of a billion in running time. Complexity analysis lets us predict that difference from the code alone, before running anything, and independently of the machine. This unit gives the notation (asymptotics), the tool for recursive algorithms (recurrences and the master theorem), the map of what is and is not feasible (P vs NP), the canonical hard problem (SAT), and the canonical easy ones (searching and sorting).

1.1 Measuring Running Time

The Model

We count **elementary operations** (comparison, assignment, arithmetic, array access—each $O(1)$ on the RAM model) as a function $T(n)$ of the input size n. We usually report the **worst case** (a guarantee), sometimes the **average case** (needs a distribution over inputs), rarely the best case. **Space complexity** counts extra memory beyond the input. Constants and lower-order terms depend on hardware and compiler, so we discard them: what remains is the *growth rate*.

Worked Example: Counting Operations in Loops

```
for i in range(n):              # n iterations
    for j in range(i):          # i iterations
        total += A[i]*A[j]      # O(1)
```

Inner body runs $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$ times $\Rightarrow \Theta(n^2)$.

```
i = 1
while i < n:                    # i = 1, 2, 4, 8, ...
    i = i * 2
```

Runs while $2^k < n$, i.e. $k < \log_2 n \Rightarrow \Theta(\log n)$. Any loop that halves or doubles is logarithmic.

```
for i in range(n):
    j = 1
    while j < n: j *= 2         # log n each
```

$\Theta(n \log n)$.

1.2 Asymptotic Notations

Formula & Mathematical Relation: The Five Notations

Let $f, g : \mathbb{N} \to \mathbb{R}^+$.

$$f(n) = O(g(n)) \iff \exists\, c > 0, n_0 :\ f(n) \le c\, g(n)\ \ \forall n \ge n_0 \qquad \text{(upper bound)} \qquad (1)$$

$$f(n) = \Omega(g(n)) \iff \exists\, c > 0, n_0 :\ f(n) \ge c\, g(n)\ \ \forall n \ge n_0 \qquad \text{(lower bound)} \qquad (2)$$

$$f(n) = \Theta(g(n)) \iff f = O(g) \text{ and } f = \Omega(g) \qquad \text{(tight bound)} \qquad (3)$$

$$f(n) = o(g(n)) \iff \lim_{n\to\infty} f(n)/g(n) = 0 \qquad \text{(strictly slower)} \qquad (4)$$

$$f(n) = \omega(g(n)) \iff \lim_{n\to\infty} f(n)/g(n) = \infty \qquad \text{(strictly faster)} \qquad (5)$$

Read O as "grows no faster than", Ω as "grows at least as fast as", Θ as "same order as".
Limit test (when the limit exists): $\lim f/g = c$ with $0 < c < \infty \Rightarrow f = \Theta(g)$; $= 0 \Rightarrow f = o(g)$;

$= \infty \Rightarrow f = \omega(g)$. L'Hôpital's rule (Unit I calculus) evaluates the limit.

Worked Example: Proving a Bound from the Definition

Show $3n^2 + 5n + 2 = O(n^2)$. For $n \geq 1$: $3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$. So $c = 10$, $n_0 = 1$ works. ■
Show $3n^2 + 5n + 2 = \Omega(n^2)$: $3n^2 + 5n + 2 \geq 3n^2$ for all $n \geq 0$; $c = 3$. Hence $\Theta(n^2)$. ■
Show $n^2 \neq O(n)$: suppose $n^2 \leq cn$ for all $n \geq n_0$; then $n \leq c$ for all large n, contradiction. ■
Limit test: $\lim \frac{n \log n}{n^2} = \lim \frac{\log n}{n} = 0$, so $n \log n = o(n^2)$.

Class	Name	Ops at $n = 10^6$	Typical algorithm
$O(1)$	constant	1	Hash lookup, array index
$O(\log n)$	logarithmic	20	Binary search, balanced BST
$O(n)$	linear	10^6	Linear search, one pass
$O(n \log n)$	linearithmic	2×10^7	Merge sort, heap sort, FFT
$O(n^2)$	quadratic	10^{12}	Bubble/insertion sort, naive pair check
$O(n^3)$	cubic	10^{18}	Naive matrix multiply, Floyd–Warshall
$O(2^n)$	exponential	10^{301030}	Brute-force subsets, naive Fibonacci
$O(n!)$	factorial	astronomically large	Brute-force permutations (TSP)

Table 1: Growth hierarchy: $1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$. A 10^9 ops/s machine does n^2 for $n = 10^6$ in 17 minutes and 2^n for $n = 60$ in 36 years.

Exam Tip & Key takeaway: Rules You Will Use Constantly

- Drop constants and lower-order terms: $5n^3 + 100n^2 + 7 = \Theta(n^3)$.
- Sum rule (sequential code): $O(f) + O(g) = O(\max(f, g))$. Product rule (nested): $O(f) \cdot O(g) = O(fg)$.
- Logarithm base does not matter: $\log_a n = \log_b n / \log_b a$, a constant factor. But 2^n vs 3^n *does* matter.
- Polynomial beats polylogarithmic: $\log^k n = o(n^\varepsilon)$ for any k, $\varepsilon > 0$. Exponential beats polynomial: $n^k = o(2^n)$.
- O is not "worst case" and Ω is not "best case"—they are bounds on *any* function. Insertion sort is $O(n^2)$ *and* $\Omega(n)$ in general, $\Theta(n^2)$ in the worst case, $\Theta(n)$ in the best case.
- $O(n^2)$ is technically true of a linear algorithm (it is an upper bound); say Θ when you mean tight.

1.3 Recurrence Relations

Definition 1.1. A **recurrence** defines $T(n)$ in terms of T at smaller arguments, plus base cases. Recursive algorithms produce them directly: (number of subproblems) $\times$ T(subproblem size) + (work to split and combine).

Reading a Recurrence Off Code

- Binary search: one half, $O(1)$ work $\Rightarrow T(n) = T(n/2) + 1$.
- Merge sort: two halves, linear merge $\Rightarrow T(n) = 2T(n/2) + n$.
- Naive recursive Fibonacci: $T(n) = T(n-1) + T(n-2) + 1$.
- Tower of Hanoi: $T(n) = 2T(n-1) + 1$.
- Selection sort written recursively: $T(n) = T(n-1) + n$.
- Strassen matrix multiply: $T(n) = 7T(n/2) + n^2$.

Three Solution Methods

1. **Iteration (unrolling):** substitute the recurrence into itself until a pattern appears, then sum.
2. **Recursion tree:** draw the calls; total = sum of work over all levels. Good for guessing.
3. **Substitution:** guess the answer, prove it by induction (Unit I). This is the only method that constitutes a *proof*.

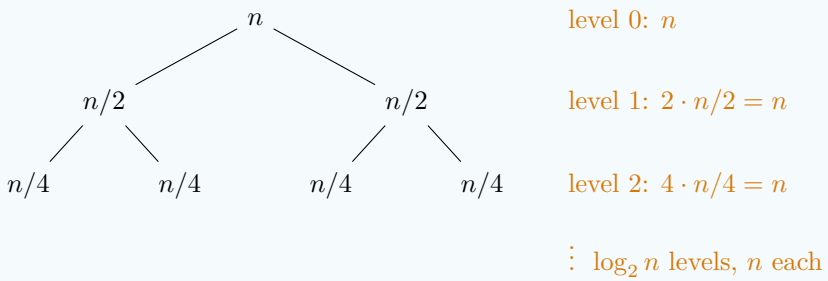
Worked Example: Iteration: $T(n) = T(n-1) + n$, $T(1) = 1$

$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \cdots = T(1) + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2)$.

Worked Example: Iteration: Tower of Hanoi $T(n) = 2T(n-1) + 1$, $T(1) = 1$

$T(n) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1 = 4T(n-2) + 2 + 1 = \cdots = 2^k T(n-k) + (2^k - 1)$. At $k = n-1$: $T(n) = 2^{n-1} \cdot 1 + 2^{n-1} - 1 = 2^n - 1 = \Theta(2^n)$. With 64 discs and one move per second: $2^{64} - 1$ s ≈ 585 billion years.

Worked Example: Recursion Tree: $T(n) = 2T(n/2) + n$



Every level does total work n; there are $\log_2 n + 1$ levels (the leaves are $T(1)$). Total $= n(\log_2 n + 1) = \Theta(n \log n)$.
Substitution proof: guess $T(n) \le cn \log_2 n$. Inductive step: $T(n) = 2T(n/2) + n \le 2c\frac{n}{2} \log_2 \frac{n}{2} + n = cn \log_2 n - cn + n \le cn \log_2 n$ whenever $c \ge 1$. ■

Formula & Mathematical Relation: Linear Recurrences with Constant Coefficients

For $a_n = c_1 a_{n-1} + c_2 a_{n-2}$, try $a_n = r^n$: the **characteristic equation** is $r^2 - c_1 r - c_2 = 0$ with roots r_1, r_2.

- Distinct roots: $a_n = \alpha r_1^n + \beta r_2^n$.
- Repeated root r: $a_n = (\alpha + \beta n) r^n$.

α, β from the initial conditions. **Fibonacci** $F_n = F_{n-1} + F_{n-2}$: $r^2 - r - 1 = 0$, $r = \frac{1 \pm \sqrt{5}}{2}$, giving Binet's formula $F_n = \frac{1}{\sqrt{5}}(\varphi^n - \psi^n)$ with $\varphi \approx 1.618$. Hence $F_n = \Theta(\varphi^n)$ and naive recursive Fibonacci is exponential.

Worked Example: Solve $a_n = 5a_{n-1} - 6a_{n-2}$, $a_0 = 1$, $a_1 = 4$

$r^2 - 5r + 6 = 0 \Rightarrow (r-2)(r-3) = 0$. $a_n = \alpha 2^n + \beta 3^n$. a_0: $\alpha + \beta = 1$. a_1: $2\alpha + 3\beta = 4$. Subtract twice the first: $\beta = 2$, $\alpha = -1$. So $a_n = 2 \cdot 3^n - 2^n$. Check $a_2 = 18 - 4 = 14 = 5(4) - 6(1)$. ✓

1.4 The Master Theorem

Formula & Mathematical Relation: Master Theorem

For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, let $d = \log_b a$ (so n^d is the total work of the leaves). Compare $f(n)$ with n^d:

Case 1 If $f(n) = O(n^{d-\varepsilon})$ for some $\varepsilon > 0$ (leaves dominate): $T(n) = \Theta(n^d)$.

Case 2 If $f(n) = \Theta(n^d)$ (every level equal): $T(n) = \Theta(n^d \log n)$.

Case 3 If $f(n) = \Omega(n^{d+\varepsilon})$ (root dominates) *and* the regularity condition $af(n/b) \leq kf(n)$ holds for some $k < 1$: $T(n) = \Theta(f(n))$.

Intuition: the recursion tree has $\log_b n$ levels; level i has a^i nodes each doing $f(n/b^i)$ work; the leaves number $a^{\log_b n} = n^{\log_b a}$. The theorem just asks which level wins.

Worked Example: One Example per Case

1. $T(n) = 9T(n/3) + n$: $a = 9, b = 3, d = \log_3 9 = 2$. $f = n = O(n^{2-1})$. **Case 1:** $\Theta(n^2)$.
2. $T(n) = T(2n/3) + 1$: $a = 1, b = 3/2, d = 0$, $n^0 = 1$. $f = 1 = \Theta(n^0)$. **Case 2:** $\Theta(\log n)$.
3. $T(n) = 3T(n/4) + n \log n$: $d = \log_4 3 \approx 0.79$. $f = n \log n = \Omega(n^{0.79+0.2})$. Regularity: $3 \cdot \frac{n}{4} \log \frac{n}{4} \leq \frac{3}{4} n \log n$ ✓. **Case 3:** $\Theta(n \log n)$.

Familiar ones: binary search $T(n) = T(n/2) + 1 \Rightarrow \Theta(\log n)$ [Case 2]; merge sort $2T(n/2) + n \Rightarrow \Theta(n \log n)$ [Case 2]; Strassen $7T(n/2) + n^2$: $d = \log_2 7 \approx 2.81$, $n^2 = O(n^{2.81-\varepsilon}) \Rightarrow \Theta(n^{2.81})$ [Case 1]; Karatsuba $3T(n/2) + n \Rightarrow \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$ [Case 1].

Common Pitfall & Misconception: When the Master Theorem Does Not Apply

- $T(n) = 2T(n/2) + n \log n$: $d = 1$, but $n \log n$ is bigger than n only by a *log* factor, not a polynomial n^ε. Falls in the gap between Cases 2 and 3. (Extended theorem: if $f = \Theta(n^d \log^k n)$ then $T = \Theta(n^d \log^{k+1} n)$, so $\Theta(n \log^2 n)$.)
- $T(n) = T(n-1) + n$: subtractive, not divisive—use iteration.
- $T(n) = 2^n T(n/2) + n$: a is not a constant.
- $T(n) = T(n/3) + T(2n/3) + n$: unequal splits; use a recursion tree ($\Theta(n \log n)$).

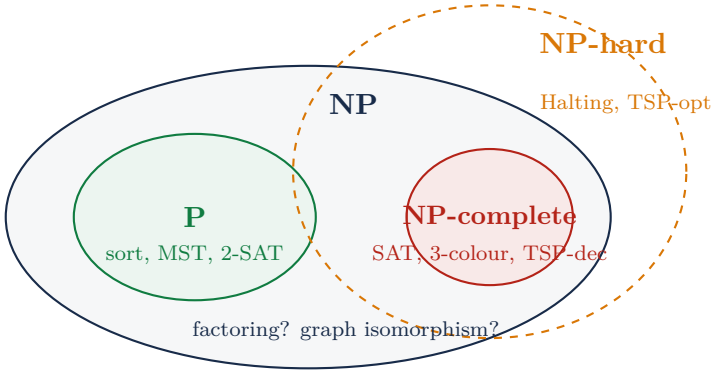
1.5 P, NP and NP-Completeness

Decision Problems and the Classes

We classify **decision problems** (yes/no answers; an optimisation problem "find the shortest tour" becomes "is there a tour of length $\leq k$?").

- **P**: solvable by a deterministic algorithm in polynomial time $O(n^k)$. Sorting, shortest paths, max flow, primality (AKS, 2002), 2-SAT, linear programming.
- **NP** (nondeterministic polynomial): a proposed *yes*-answer (**certificate**) can be *verified* in polynomial time. Equivalently, solvable in polynomial time by a machine that can guess. Every problem in P is in NP (just re-solve it to verify): P $\subseteq$ NP.
- **NP-hard**: at least as hard as every problem in NP—every NP problem *reduces* to it in polynomial time. Need not itself be in NP (e.g. the halting problem, or optimisation versions).
- **NP-complete**: in NP *and* NP-hard. The "hardest problems in NP". If any one of them is in P, then P = NP.

The open question: is P = NP? Almost universally believed *no*, unproven; a Clay Millennium Prize problem.



Formula & Mathematical Relation: Polynomial-Time Reduction

$A \leq_p B$: there is a polynomial-time function r such that x is a yes-instance of A iff $r(x)$ is a yes-instance of B. Consequences: if $B \in \mathrm{P}$ then $A \in \mathrm{P}$ ("B is at least as hard as A"). To prove a new problem X is NP-complete:

1. Show $X \in \mathrm{NP}$ (give a polynomial verifier).
2. Pick a known NP-complete problem Y and show $Y \leq_p X$.

Cook–Levin theorem (1971): SAT is NP-complete—the first one, proved directly from the definition. Karp (1972) then reduced SAT to 21 other problems.

NP-complete problem	Question	Easy cousin (in P)
SAT / 3-SAT	Is there a satisfying assignment?	2-SAT, Horn-SAT
Vertex cover	Is there a set of $\leq k$ vertices touching every edge?	On bipartite graphs (Kőnig)
Clique / Independent set	Is there a complete subgraph (or edgeless set) of size $\geq k$?	—
Hamiltonian cycle / TSP	Cycle through all vertices? Tour of length $\leq k$?	Eulerian circuit
Graph 3-colouring	Is $\chi(G) \leq 3$?	2-colouring (bipartite test)
Subset sum / Knapsack	Subset summing to exactly k?	Pseudo-polynomial DP $O(nW)$
Partition / Bin packing	Split into equal halves? Pack into k bins?	—

Table 2: Canonical NP-complete problems. Each pair of "hard / easy cousin" shows how a small change in the problem statement crosses the boundary.

Exam Tip & Key takeaway: What NP-Completeness Means for an Engineer

An NP-complete label does **not** mean "give up". It means no known algorithm is polynomial in the worst case, so you choose among: (i) **exact** exponential algorithms with pruning (backtracking, branch and bound, Unit VI) for small n; (ii) **approximation** algorithms with guaranteed ratio (2-approx vertex cover, 1.5-approx metric TSP); (iii) **heuristics / metaheuristics** (greedy, genetic algorithms, Unit VI); (iv) **special cases** that are in P (trees, bipartite, bounded treewidth); (v) **SAT/ILP solvers**, which handle many practical instances with millions of variables despite worst-case exponential behaviour.

1.6 The Satisfiability Problem (SAT)

Definition 1.2. A Boolean formula in **conjunctive normal form (CNF)** is an AND of **clauses**, each an OR of **literals** (a variable or its negation). **SAT** asks: is there an assignment of true/false to the variables making the formula true? **k-SAT** restricts every clause to exactly k literals.

Where the Boundary Lies

- **2-SAT** is in P: build the implication graph (clause $(a \vee b)$ gives $\neg a \to b$ and $\neg b \to a$); unsatisfiable iff some x and $\neg x$ are in the same strongly connected component. $O(n+m)$ with DFS.
- **3-SAT** is NP-complete (any CNF converts to 3-CNF with polynomial blow-up). 3-SAT is the standard starting point for reductions.
- **Horn-SAT** (each clause has at most one positive literal—logic programming / Prolog) is in P by unit propagation.
- Brute force: 2^n assignments $\times$ $O(m)$ to check. For $n = 100$ that is 10^{30}.

Worked Example: Deciding Satisfiability by Hand

$\Phi = (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee \neg z) \wedge (x \vee \neg z)$.
Try $x = \mathrm{T}$: clauses 1 and 4 satisfied. Clause 2 forces $z = \mathrm{T}$. Clause 3 then forces $y = \mathrm{F}$. Assignment $(x, y, z) = (\mathrm{T}, \mathrm{F}, \mathrm{T})$: check $(\mathrm{T}) \wedge (\mathrm{T}) \wedge (\mathrm{T} \vee \mathrm{F}) \wedge (\mathrm{T}) = \mathrm{T}$. **Satisfiable.**
$\Psi = (x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee y) \wedge (\neg x \vee \neg y)$. Resolution on clauses 1, 2 (on y): x. On 3, 4: $\neg x$. Resolving x with $\neg x$ gives the empty clause. **Unsatisfiable.** (Also visible: the four clauses rule out all four assignments, one each.)

Formula & Mathematical Relation: DPLL: How SAT Solvers Actually Work

1. **Unit propagation:** a clause with a single unassigned literal forces that literal.
2. **Pure literal elimination:** a variable appearing with only one polarity can be set to satisfy all its clauses.
3. **Branch:** pick an unassigned variable, try T; on conflict, backtrack and try F.

Modern CDCL solvers add *clause learning* from conflicts and *non-chronological backjumping*. This is backtracking (Unit VI) plus the resolution rule of inference (Unit I).

Where Computer Science Uses This: SAT in Practice

Hardware verification (does this circuit equal that one? — assert they differ and check unsatisfiability), software model checking, package dependency resolution (`apt`, `conda`), automated planning, cryptanalysis, and timetabling all get encoded into CNF and handed to a SAT solver. The theoretical hardness is worst-case; structured industrial instances are usually solved fast.

1.7 Searching and Sorting

1.7.1 Searching

Linear vs. Binary Search

- **Linear search**: scan until found. Worst and average $\Theta(n)$. Works on unsorted data.
- **Binary search**: requires sorted data. Compare with the middle; discard half. $T(n) = T(n/2) + 1 = \Theta(\log n)$. At most $\lfloor \log_2 n \rfloor + 1$ comparisons.

```
def binary_search(A, key):
    lo, hi = 0, len(A) - 1
    while lo <= hi:
        mid = (lo + hi) // 2          # lo + (hi - lo)//2 avoids
   overflow in C
        if A[mid] == key: return mid
```

```
        elif A[mid] < key: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Worked Example: Binary Search Trace

$A = [3, 8, 12, 17, 23, 31, 40, 48, 56]$, key $= 31$.

lo	hi	mid	A[mid]	action
0	8	4	23	$23 < 31 \Rightarrow$ lo $= 5$
5	8	6	40	$40 > 31 \Rightarrow$ hi $= 5$
5	5	5	31	found at index 5

Three comparisons for $n = 9$; linear search would need six. For $n = 10^9$, binary search needs at most 30.

Lower bound: any comparison-based search on n sorted items needs $\Omega(\log n)$ comparisons—a decision tree with n leaves has height $\geq \log_2 n$. So binary search is optimal.

1.7.2 Sorting

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble	n	n^2	n^2	1	yes	Teaching only; early exit if no swaps
Selection	n^2	n^2	n^2	1	no	Fewest swaps $(n-1)$
Insertion	n	n^2	n^2	1	yes	Best for small or nearly-sorted input; online
Merge	$n \log n$	$n \log n$	$n \log n$	n	yes	Guaranteed; external sorting; linked lists
Quick	$n \log n$	$n \log n$	n^2	$\log n$	no	Fastest in practice; worst case on sorted input with bad pivot
Heap	$n \log n$	$n \log n$	$n \log n$	1	no	In-place guarantee; priority queues
Counting	$n + k$	$n + k$	$n + k$	$n + k$	yes	Integers in $[0, k]$; not comparison-based
Radix	$d(n + k)$	$d(n + k)$	$d(n + k)$	$n + k$	yes	d digits; uses counting sort per digit

Table 3: Sorting algorithms (Θ bounds, "$n \log n$" means $\Theta(n \log n)$). *Stable* = equal keys keep their original order. Python's `sorted` is Timsort (merge + insertion, stable, $O(n \log n)$); C++ `std::sort` is introsort (quick + heap + insertion).

Formula & Mathematical Relation: The $\Omega(n \log n)$ Lower Bound for Comparison Sorting

A comparison sort is a decision tree: each internal node compares two elements, each leaf is one of the $n!$ possible output orderings. A binary tree with $n!$ leaves has height $h \geq \log_2(n!)$. By Stirling (or the sum–integral bound from Unit I),

$$\log_2(n!) = \sum_{k=1}^{n} \log_2 k \;\geq\; \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n).$$

So merge sort and heap sort are asymptotically optimal among comparison sorts. Counting and radix sort beat the bound only because they do not compare—they inspect the keys' digits, assuming bounded integers.

Worked Example: Merge Sort

Sort $[38, 27, 43, 3, 9, 82, 10]$.

Split: $[38, 27, 43, 3] \mid [9, 82, 10]$
$[38, 27] \mid [43, 3]$ $[9, 82] \mid [10]$
$[38][27]$ $[43][3]$ $[9][82]$ $[10]$
Merge: $[27, 38]$ $[3, 43]$ $[9, 82]$ $[10]$
$[3, 27, 38, 43]$ $[9, 10, 82]$
$[3, 9, 10, 27, 38, 43, 82]$

Merging two sorted lists of total length k takes $k-1$ comparisons at most. $\lceil \log_2 7 \rceil = 3$ levels of merging, ≤ 7 work each: $T(n) = 2T(n/2) + n = \Theta(n \log n)$ regardless of input order.

Worked Example: Quicksort Partition (Lomuto, pivot = last element)

$A = [7, 2, 9, 4, 6]$, pivot $= 6$. Maintain $i =$ end of the "$\leq$ pivot" zone (start $i = -1$); scan j from 0 to 3.

j	$A[j]$	≤ 6?	array after
0	7	no	$[7, 2, 9, 4, 6]$
1	2	yes: $i = 0$, swap $A[0], A[1]$	$[2, 7, 9, 4, 6]$
2	9	no	$[2, 7, 9, 4, 6]$
3	4	yes: $i = 1$, swap $A[1], A[3]$	$[2, 4, 9, 7, 6]$
end		swap pivot into $A[i+1] = A[2]$	$[2, 4, \mathbf{6}, 7, 9]$

Pivot is now at its final index 2; recurse on $[2, 4]$ and $[7, 9]$. Balanced partitions give $T(n) = 2T(n/2)+n = \Theta(n \log n)$; a pivot that is always the min or max gives $T(n) = T(n-1)+n = \Theta(n^2)$ (already-sorted input with last-element pivot). Randomised pivot or median-of-three makes the expected time $\Theta(n \log n)$ on *every* input.

Exam Tip & Key takeaway: Choosing a Sort

Small n or nearly sorted $\rightarrow$ insertion. Need stability + guarantee $\rightarrow$ merge. Memory-tight + guarantee $\rightarrow$ heap. General in-memory speed $\rightarrow$ quicksort (randomised). Bounded integers / fixed-width keys $\rightarrow$ counting or radix. Data on disk $\rightarrow$ external merge sort. Only need the top $k \rightarrow$ heap ($O(n \log k)$) or quickselect ($O(n)$ expected).

1.8 Unit V Quick-Revision Sheet

Exam Tip & Key takeaway: Ten Things to Carry Into the Exam

1. O = upper, Ω = lower, Θ = tight; prove with explicit c, n_0 or the limit test.
2. Hierarchy: $1 \prec \log n \prec n \prec n \log n \prec n^2 \prec 2^n \prec n!$. Halving loop $= \log n$; nested loops multiply.
3. Recurrence from code: (subproblems) $\times\, T$(size) + (split/merge work). Solve by iteration, recursion tree, or substitution (= induction).
4. $T(n-1) + n \rightarrow n^2$; $2T(n-1) + 1 \rightarrow 2^n$; $T(n/2) + 1 \rightarrow \log n$; $2T(n/2) + n \rightarrow n \log n$.
5. Master theorem: compare $f(n)$ with $n^{\log_b a}$. Smaller by $n^{\varepsilon} \rightarrow \Theta(n^{\log_b a})$; equal $\rightarrow \times \log n$; larger by n^{ε} (+ regularity) $\rightarrow \Theta(f)$.
6. Characteristic equation for $a_n = c_1 a_{n-1} + c_2 a_{n-2}$; Fibonacci grows as φ^n.
7. P: solve in poly time. NP: *verify* in poly time. NP-complete: in NP and everything in NP reduces to it. P $\subseteq$ NP; equality open.
8. Prove NP-completeness: (1) in NP, (2) reduce a known NP-complete problem *to* it. Cook–Levin: SAT was first.

9. 2-SAT $\in$ P (implication graph); 3-SAT NP-complete. DPLL = unit propagation + pure literals + backtracking.
10. Binary search $\Theta(\log n)$, needs sorted input. Comparison sorting is $\Omega(n \log n)$; merge/heap achieve it; quicksort $\Theta(n^2)$ worst but fastest on average; counting/radix are linear for bounded keys.